

Методичка: Docker, Nginx-контейнер, volume, Dockerfile и Compose

Темы: Docker Engine, image, container, registry, ports, env, volumes, bind mounts, logs, exec, Dockerfile, Compose

ОС сервера: Debian Server 12/13 или Ubuntu Server 22.04/24.04 LTS

ОС клиента: Debian/Ubuntu или Windows 10/11, если используется

Среда: VirtualBox

Формат: самостоятельная практическая работа

Ориентировочное время: 4 академических часа

1. Что настроим

В этой работе разворачиваем или проверяем серверный сервис Linux-инфраструктуры. Команды выполняем на Debian/Ubuntu, а результат подтверждаем локально и с другого узла.

Используем узлы: docker01, client01.

Главная идея работы:

Контейнеризация считается настроенной не тогда, когда контейнер запущен, а тогда, когда опубликованный порт отвечает, данные сохраняются через volume/bind mount, логи читаются, а compose-файл позволяет воспроизвести запуск.

2. Схема учебного стенда

Узел	Роль	ОС	Сетевые адаптеры	IP-адрес
srv01	основной сервер сервиса	Debian/Ubuntu Server	NAT + внутренняя сеть	192.168.10.10/24
srv02	второй сервер или клиент	Debian/Ubuntu Server	NAT + внутренняя сеть	192.168.10.20/24
client01	клиентская проверка	Debian/Ubuntu или Windows	внутренняя сеть	192.168.10.30/24

VirtualBox NAT для пакетов

```
|
srv01 ---- linux-lab 192.168.10.0/24 ---- srv02/client01
```

Если используем имена, добавляем DNS или `/etc/hosts` и проверяем `getent hosts`.

3. Необходимые ресурсы и предварительные условия

Нужны:

- Debian/Ubuntu Server с NAT;
- пакеты Docker из репозитория ОС или Docker repo;
- каталог `/srv/docker/nginx-html`;

- клиент для проверки HTTP на порту 8080.

4. Подготовка виртуальных машин

```
hostnamectl  
ip -br address  
ip route  
resolvectl status  
systemctl --failed
```

Проверяем имена:

```
getent hosts srv01.company.test  
getent hosts srv02.company.test
```

Если имена не находятся, добавляем записи в `/etc/hosts`.

Самопроверка этапа

- NAT работает для установки пакетов;
- внутренняя сеть доступна;
- имена узлов разрешаются;
- снимки ВМ созданы.

Если результат отличается от ожидаемого, дальше пока не продолжаем. Сначала проверяем этот этап.

5. Адресный план или план ресурсов

Ресурс	Значение
Основной сервер	srv01, 192.168.10.10

Второй сервер	srv02, 192.168.10.20
Клиент	client01, 192.168.10.30
Учебный домен	company.test
Данные сценария	по теме лекции

План работы:

- установить Docker;
- запустить Nginx-контейнер на порту 8080;
- подключить HTML через bind mount;
- проверить logs и exes;
- настроить restart policy;
- пересоздать контейнер с сохранением данных;
- создать Dockerfile и compose.yaml.

6. Исходная проверка

```
cat /etc/os-release
ip -br address
ip route
ss -lntup
systemctl --failed
df -h
```

Самопроверка этапа

- ОС и сеть проверены;

- нужные порты свободны или понятно, какая служба их использует;
- свободного места достаточно;
- failed-службы разобраны до настройки.

7. Пошаговая настройка

7.1. Подготовка пакетов

```
sudo apt update
```

Если пакеты не скачиваются, проверяем NAT, DNS и маршрут.

7.2. Основная настройка

```
sudo apt install docker.io docker-compose-plugin -y
sudo systemctl enable --now docker
sudo mkdir -p /srv/docker/nginx-html
echo 'Hello from Docker nginx' | sudo tee /srv/docker/nginx-html/index.html
sudo docker run -d --name web-nginx --restart unless-stopped -p 8080:80 -v
/srv/docker/nginx-html:/usr/share/nginx/html:ro nginx:stable
sudo docker ps
curl -I http://127.0.0.1:8080
sudo docker logs web-nginx
sudo docker exec web-nginx nginx -v
```

Создаём `compose.yaml`:

```
mkdir -p ~/docker-nginx
nano ~/docker-nginx/compose.yaml

services:
  web:
    image: nginx:stable
    container_name: web-nginx-compose
    restart: unless-stopped
    ports:
      - "8081:80"
    volumes:
      - /srv/docker/nginx-html:/usr/share/nginx/html:ro
```

Самопроверка этапа

```
sudo docker ps
sudo docker inspect web-nginx --format '{{.HostConfig.RestartPolicy.Name}}'
curl http://127.0.0.1:8080
cd ~/docker-nginx && sudo docker compose config
cd ~/docker-nginx && sudo docker compose up -d
```

```
curl http://127.0.0.1:8081
```

Если результат отличается от ожидаемого, дальше пока не продолжаем.

Сначала проверяем этот этап.

7.3. Восстановление или откат

Для backup, мониторинга, Docker и HAProxy обязательно проверяем восстановление или повторное создание результата: восстановить файл, вернуть сервис, пересоздать контейнер с сохранением данных или вернуть backend в балансировку.

8. Проверка итогового результата

```
sudo docker ps
sudo docker inspect web-nginx --format '{{.HostConfig.RestartPolicy.Name}}'
curl http://127.0.0.1:8080
cd ~/docker-nginx && sudo docker compose config
cd ~/docker-nginx && sudo docker compose up -d
curl http://127.0.0.1:8081
```

Границы результата:

- учебный стенд не заменяет промышленную отказоустойчивую архитектуру;
- тестовые пароли и ключи нельзя переносить в реальную среду;
- сервис считается готовым только после клиентской проверки, журнала и проверки восстановления, если она относится к теме.

9. Диагностика типовых ошибок

Симптом	Вероятная причина	Проверка	Исправление
Порт не открывается	Контейнер не запущен или порт занят	<code>docker ps</code> , <code>ss -lntup</code>	Исправить port mapping
Данные пропали	Контейнер был без volume	<code>docker inspect</code> , <code>ls /srv/docker</code>	Использовать volume/bind mount
Compose не стартует	Ошибка YAML	<code>docker compose config</code>	Исправить отступы и ключи
Пакет не устанавливается	Нет доступа к репозиториям	<code>ip route</code> , <code>getent hosts deb.debian.org</code>	Проверить NAT и DNS
Сервис не отвечает	Порт не слушается или firewall блокирует	<code>ss -lntup</code> , <code>nc -vz</code>	Проверить службу и firewall

Места на диске мало	Логи, backup или volumes растут	<code>df -h, du -xhd1</code>	Очистить или расширить хранилище по плану
------------------------	---------------------------------------	------------------------------	--

10. Итоговая самостоятельная проверка

- ☐ Стенд соответствует схеме.
- ☐ Имена и адреса согласованы.
- ☐ Пакеты установлены.
- ☐ Сервис или сценарий запущен.
- ☐ Проверочные команды подтверждают результат.
- ☐ Клиентская проверка проходит.
- ☐ Восстановление или повторное создание результата проверено.
- ☐ В журналах нет необъяснённых ошибок.
- ☐ Одна типовая ошибка найдена и исправлена.

11. Что приложить к отчёту

К отчёту прикладываем схему, адресный план, ключевой фрагмент конфигурации, вывод проверки, клиентский результат, журнал/метрики, описание исправленной ошибки и результат восстановления.

12. Контрольные вопросы

1. Какую задачу решает технология этой лекции?
2. Какие ресурсы нужно оценить до внедрения?
3. Какая команда подтверждает основной результат?
4. Как проверить сервис с клиента?
5. Что проверить в журналах?
6. Чем учебная настройка отличается от промышленной?
7. Как выполнить безопасный откат?
8. Что приложить к отчёту?